# Summer School in AI & Applied Economics

Text and Image as Data — Problem Set

Week 1, 31 August – 4 September 2026

The goal of this problem set is to give you hands-on experience with the methods in text analysis and computer vision that are relevant for social science research.

**Instructions:** This is a mini-project. If you want, you are free to develop your own research idea involving tools in text or image analysis. Please contact Andrea Ciccarone if you intend to work on your own research idea. You should aim to:

- Define the research question and objectives
- Obtain, clean, and describe the data
- Produce some initial result. This does not have to be a clear-cut result. Observations on patterns, analysis of descriptive statistics or simple correlations is enough
- Indicate directions for future development

## Data

You are free to work on your own research idea, which **has to be centered on image and text analysis**. If you prefer a more guided approach, use the dataset provided with this problem set, `news_videos.csv.gz`. It contains **23,689 videos** posted to YouTube by two US cable news outlets — **Fox News** (11,540) and **MSNBC** (12,149) — covering **January to August 2026**, with one row per video:

1. `video_id`, `url` — identifier and link
2. `outlet`, `channel_id` — which outlet posted it
3. `title`, `description` — the text
4. `published_at` — date and time of posting
5. `view_count`, `like_count`, `comment_count` — engagement
6. `duration_seconds` — video length
7. `days_since_publication` — how long the video has been online

8. `thumbnail_url` — the image the outlet chose to represent the video

The two outlets post at a similar rate — between about 1,350 and 1,700 videos each per month — so the panel is close to balanced in every month of the sample. This gives you eight months of within-outlet variation, which means you can look at how coverage and engagement move over time, not just at cross-sectional differences.

Views accumulate, so a raw view count is not comparable across videos of different ages. That is why `days_since_publication` is included, and why you should control for it in anything that uses engagement as an outcome.

The thumbnail is the object of interest on the image side: it is chosen by the outlet, it is the first thing a viewer sees, and it is the visual analogue of a headline. Images are fetched directly from the URL, so nothing needs to be downloaded in advance:

```
import pandas as pd, requests
from PIL import Image
from io import BytesIO

df = pd.read_csv("news_videos.csv.gz")   # pandas decompresses
    transparently
img = Image.open(BytesIO(requests.get(df.thumbnail_url[0]).content)).
    convert("RGB")
```

The file is gzipped to keep it small; `pandas` reads it directly. If you want to rebuild or extend the dataset, `build_dataset.py` collects it from scratch and lets you change the outlets or the time window. Note that 23,689 thumbnails is a lot to embed on a free Colab GPU — start with a random subsample of a few thousand, get the pipeline working, and scale up only if you need to.

# What to do

You can do all or some of the following. Please feel free to experiment.

- Before you analyse your images, you need to preprocess them (resize, crop, and so on) to prepare them for a model. Most modern encoders ship with the exact preprocessing they were trained with, so prefer that over rolling your own:

  ```
  import torchvision.transforms as transforms

  preprocess = transforms.Compose([
      transforms.Resize(256),
      transforms.CenterCrop(224),
      transforms.ToTensor(),
      transforms.Normalize(mean=[0.485, 0.456, 0.406],
                           std=[0.229, 0.224, 0.225]),
  ])
  ```

- **Represent images as vectors.** The old way was to take a network trained for classification and strip its last layer. That still works, but you will get a better representation from a model trained to align images with language. Use **CLIP**:

```
import open_clip, torch

model, _, preprocess = open_clip.create_model_and_transforms(
    "ViT-B-32", pretrained="laion2b_s34b_b79k")
model.eval()

with torch.no_grad():
    v = model.encode_image(preprocess(img).unsqueeze(0))
    v = v / v.norm(dim=-1, keepdim=True)   # normalise, then cosine =
        dot product
```

CLIP runs locally and is free. A hosted multimodal model such as **Gemini** will usually give you a stronger embedding, at a small cost per image and with an API key; if you use one, embed a subsample first and check the cost before running the full dataset. Either way, **fix the model and write down which version you used**: embeddings from different models are not comparable.

- **Use the representation to cluster images** (PCA, $k$-means, and so on). Are there partisan differences in representations across outlets? Do the clusters correspond to anything you can name after reading the videos nearest each centroid?

- **Use your vector representation to train a prediction model** (logistic regression, random forest, gradient boosting, a small neural network). Two natural targets:

  1. **Predict the outlet** from the thumbnail alone. How well can a classifier tell Fox from MSNBC? The classes are balanced by construction, so accuracy is readable, but report AUC on a held-out set as well.
  2. **Predict engagement** — views, likes, or comments — from the image, the text, or both. Engagement is heavily skewed, so consider modelling $\log(1 + y)$, and control for `days_since_publication` and `duration_seconds`.

  Fit the same model on text features, on image features, and on both, and compare. If the joint model beats each single-modality model, the two carry different information.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

Xtr, Xte, ytr, yte = train_test_split(V, y, test_size=0.3,
                                      stratify=y, random_state=0)
clf = LogisticRegression(max_iter=2000).fit(Xtr, ytr)
print(roc_auc_score(yte, clf.predict_proba(Xte)[:, 1]))
```

**A warning that matters here.** Outlets brand their thumbnails: logos, chyrons, recurring studio sets, house fonts. A classifier that separates Fox from MSNBC at AUC 0.99 has probably learned to find a logo, not a political style. Look at what drives your predictions before you believe them — inspect the images the model is most and least confident about, and consider cropping or masking the branded region and checking whether performance survives.

- Because CLIP puts images and text in the *same* space, you can also **score an image against a sentence** without any training data:

```
tokenizer = open_clip.get_tokenizer("ViT-B-32")
prompts = ["a photo of a protest", "a photo of a politician speaking"
    ]
with torch.no_grad():
    t = model.encode_text(tokenizer(prompts))
    t = t / t.norm(dim=-1, keepdim=True)
scores = (v @ t.T).squeeze()
```

  This is a cheap way to build a continuous measure of image content. The prompt is a research choice: report it, and check that your result survives reasonable rewordings.

- You can use off-the-shelf packages to **detect faces** and keep only thumbnails containing them. Useful Python libraries: `face_recognition`, `DeepFace`, `OpenCV`. For those images you can then detect facial attributes; gender and age detection are the easiest to implement, and there are many pre-trained models available.

- Once you have labelled images, you can start working on the text.

**Important note on off-the-shelf packages.** The packages mentioned here may not be reliable when applied to tasks different from the ones they were built for. Expect their predictions to be imprecise, and validate them on a subsample you have looked at yourself.

**Should you fine-tune your own model?** You are of course free to train your own model or fine-tune a pre-trained one. There are many good tutorials online. It may not be easy to learn this in a couple of weeks, so fine-tuning is not necessary. Just know that you lose some reliability with off-the-shelf pre-trained models. I am available to help if you want to fine-tune your own.

**For text analysis:** feel free to implement whatever method you want. Most methods have a Python library and are easy to find. You can use methods based on bag-of-words representations, and I also suggest you experiment with embeddings. The modern default is a sentence embedding model rather than word vectors:

```
from sentence_transformers import SentenceTransformer
enc = SentenceTransformer("all-MiniLM-L6-v2")
E = enc.encode(df.title.tolist(), normalize_embeddings=True)
```

Possible questions you may be able to investigate: is the representation of different genders, age groups or races different across the two outlets? When Fox and MSNBC cover the same story, do they choose systematically different images? Is there any correlation between certain types of video or thumbnail and views, likes, or comments?

**What should you use?** Python. The easiest option is Google Colab, which requires no setup, is free, and gives you free compute. It is a Jupyter notebook, so it is also easy to visualise. If you are comfortable with something else, feel free to use it.